

Lecture #10

Linked Lists

Today's Discussion...

- * Introduction to linked list

- * Array versus linked list

- * Linked lists in C++

- * Types of linked lists

 - * Single linked list

 - * Doubly linked list

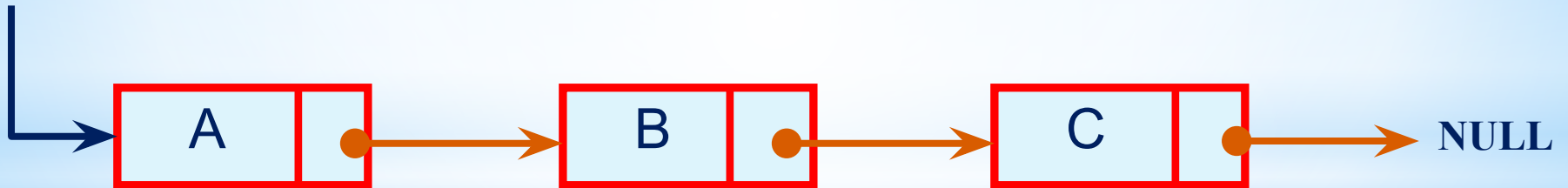
 - * Circular linked list

Introduction to Linked Lists

Linked List

- A linked list is a **data structure** which allows to store data dynamically and manage data efficiently.
- Typically, a linked list, in its simplest form looks like the following

header

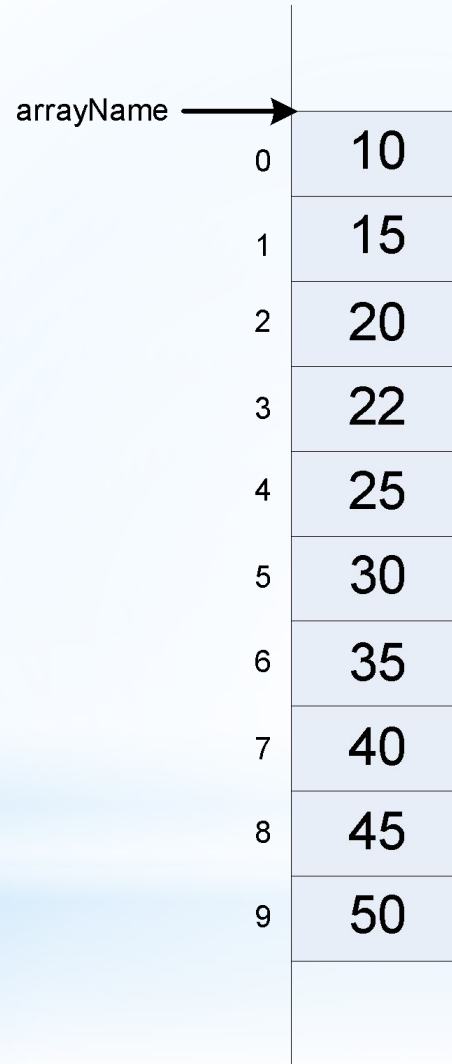


Linked List

- Few salient features
 - There is a pointer (called **header**) points the first element (also called node)
 - Successive nodes are connected by **pointers**.
 - Last element points to **NULL**.
 - It can grow or shrink in size during execution of a program.
 - It can be made just as long as required.
 - It does not waste memory space, consume exactly what it needs.

Arrays versus Linked Lists

Array: Contagious Storage



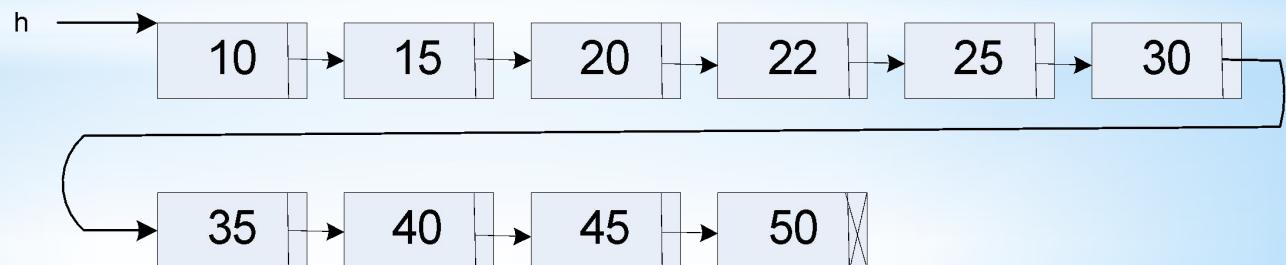
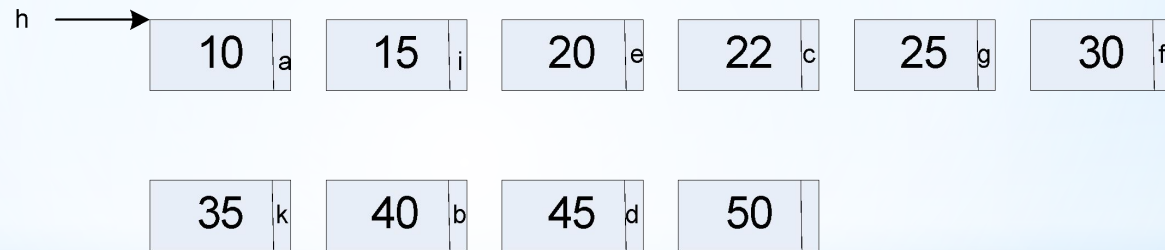
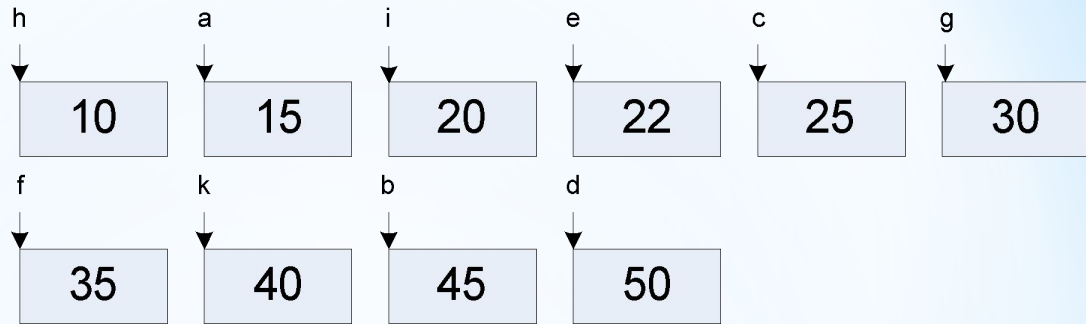
Array versus Linked Lists

- **In arrays**

- elements are stored in a contiguous memory locations
- Arrays are static data structure unless we use dynamic memory allocation
- Arrays are suitable for
 - Inserting/deleting an element at the end.
 - Randomly accessing any element.
 - Searching the list for a particular value.

Linked List: Non-Contiguous Storage

a	15
b	45
c	25
d	50
e	22
f	35
g	30
h	10
i	20
j	
k	40



Array versus Linked Lists

- **In Linked lists**

- adjacency between any two elements are maintained by means of links or pointers
- It is essentially a dynamic data structure
- Linked lists are suitable for
 - Inserting an element at any position.
 - Deleting an element from any where.
 - Applications where sequential access is required.
 - In situations, where the number of elements cannot be predicted beforehand.

Linked Lists in C

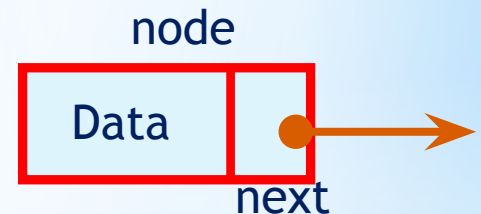
Defining a Node of a Linked List

Each structure of the list is called a **node**, and consists of two fields:

- Item (or) data
- Address of the next item in the list (or) pointer to the next node in the list

How to define a node of a linked list?

```
struct node
{
    int data;           /* Data */
    struct node *next; /* pointer */
} ;
```



Note:

Such structures which contain a member field pointing to the same structure type are called **self-referential structures**.

Types of Lists: Single Linked List

Depending on the way in which the links are used to maintain adjacency, several different types of linked lists are possible.

Single linked list (or simply linked list)

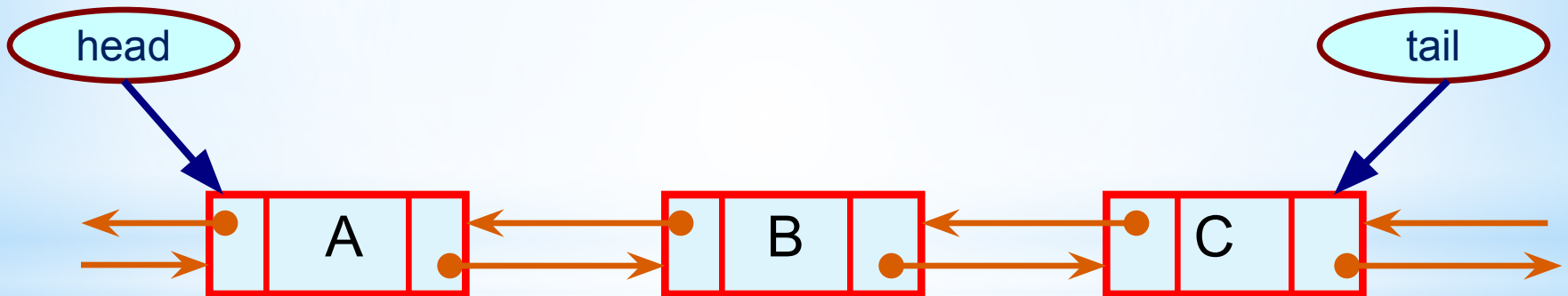
- A head pointer addresses the first element of the list.
- Each element points at a successor element.
- The last element has a link value NULL.



Types of Lists: Double Linked List

Double linked list

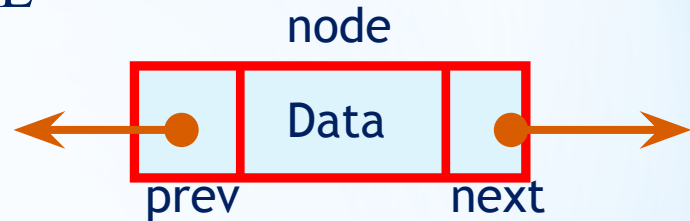
- Pointers exist between adjacent nodes in both directions.
- The list can be traversed either forward or backward.
- Usually two pointers are maintained to keep track of the list, *head* and *tail*.



Defining a Node of a Double Linked List

Each node of doubly linked list (DLL) consists of three fields:

- Item (or) Data
- Pointer of the next node in DLL
- Pointer of the previous node in DLL

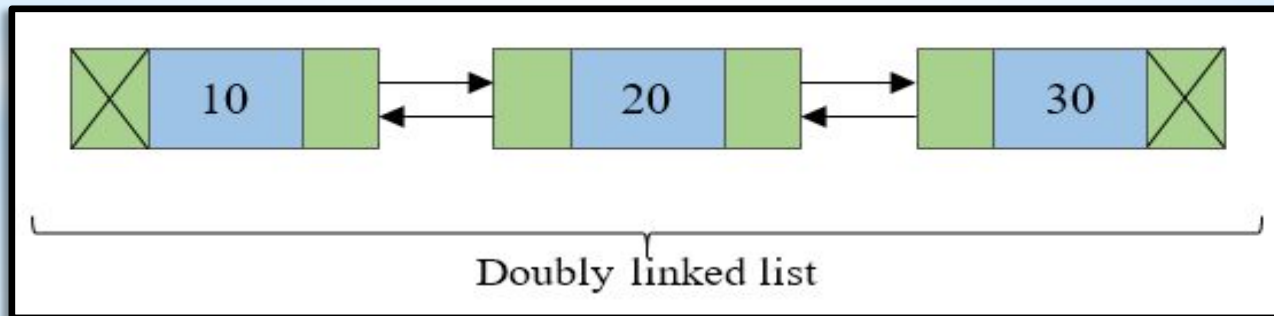


How to define a node of a doubly linked list (DLL)?

```
struct node
{
    int data;
    struct node *next; // Pointer to next node in DLL
    struct node *prev; // Pointer to previous node in DLL
};
```

Double Linked List

- Doubly linked list is a collection of nodes linked together in a sequential way.
- Doubly linked list is almost similar to singly linked list except it contains **two address or reference fields**, where one of the address field contains reference of the next node and other contains reference of the previous node.
- **First and last node** of a linked list contains a terminator generally a **NULL** value, that determines the start and end of the list.
- Doubly linked list is sometimes also referred as **bi-directional linked list** since it allows traversal of nodes in both direction.
- Since doubly linked list allows the traversal of nodes in both direction, we can keep track of both first and last nodes.



Double versus Single Linked List

Advantages over singly linked list

- 1) A DLL can be traversed in both forward and backward direction.
- 2) The delete operation in DLL is more efficient if pointer to the node to be deleted is given.

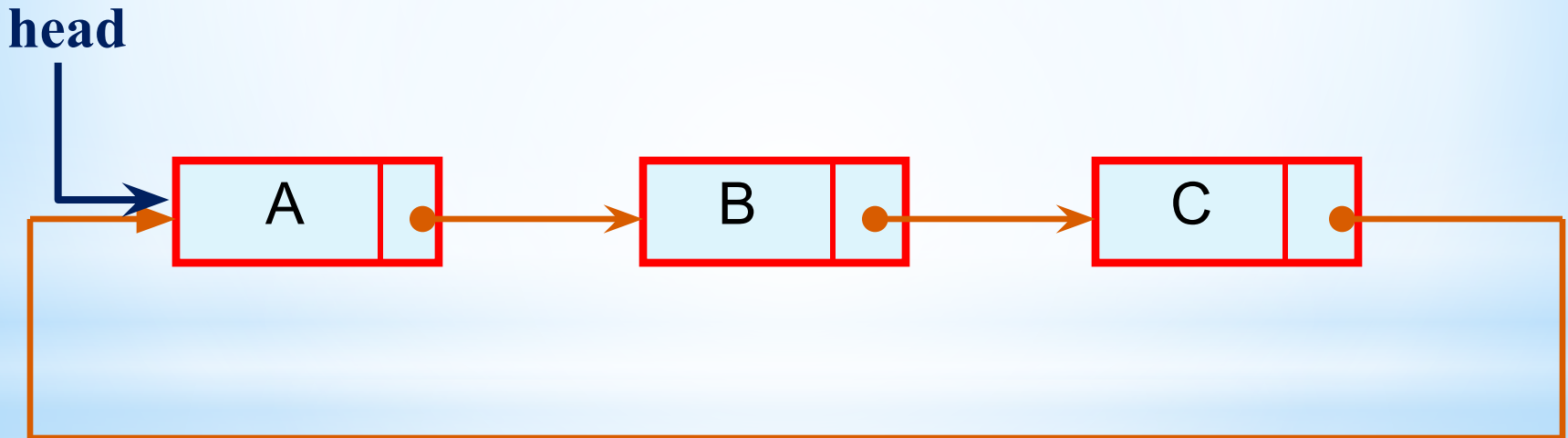
Disadvantages over singly linked list

- 1) Every node of DLL Require extra space for an previous pointer.
- 2) All operations require an extra pointer previous to be maintained.

Types of Lists: Circular Linked List

Circular linked list

- The pointer from the last element in the list points back to the first element.



Circular Linked List

- A circular linked list is basically a linear linked list that may be **single-** or **double-linked**.
- The only difference is that **there is no any NULL** value terminating the list.
- In fact in the list every node points to the next node and last node points to the first node, thus forming a circle. Since it forms a **circle** with **no end to stop** it is called as **circular linked list**.
- In circular linked list there can be no starting or ending node, whole node can be **traversed from any node**.
- In order to traverse the circular linked list, only once we need to traverse entire list until the **starting node is not traversed again**.
- A circular linked list can be implemented using both **singly linked list** and **doubly linked list**.

Example 1: Creating a Single Linked List

Linked list to store and print roll number, name and age of 3 students.

```
#include <iostream>
#include <string>

// In C++, 'struct' automatically defines a type, so you don't need 'struct stud' later
struct Stud {
    int roll;
    std::string name; // Using std::string is safer and more flexible than char arrays
    int age;
    Stud* next;
};

int main() {
    Stud n1, n2, n3;
    Stud* p = nullptr;

    std::cout << "Enter the Three students Roll no, Name, and Age: \n";
    std::cin >> n1.roll >> n1.name >> n1.age;
    std::cin >> n2.roll >> n2.name >> n2.age;
    std::cin >> n3.roll >> n3.name >> n3.age;

    n1.next = &n2;
    n2.next = &n3;
    n3.next = nullptr; // nullptr is the modern C++ standard for null pointers
```

Example 1: Creating a Single Linked List

```
/* Traverse the list and print elements along with their
memory addresses */
p = &n1;
while (p != nullptr) {
    // Printing 'p' directly outputs its hexadecimal
memory address
    std::cout << "\nNode Address: " << p
                << " | Data: " << p->roll
                << " " << p->name
                << " " << p->age;

    p = p->next;
}
std::cout << std::endl;
return 0;
}
```

Example 1: Illustration

The structure:

```
struct stud
{
    int roll;
    char name[30];
    int age;
    struct stud *next;
};
```

Also assume the list with three nodes n1, n2 and n3 for 3 students.

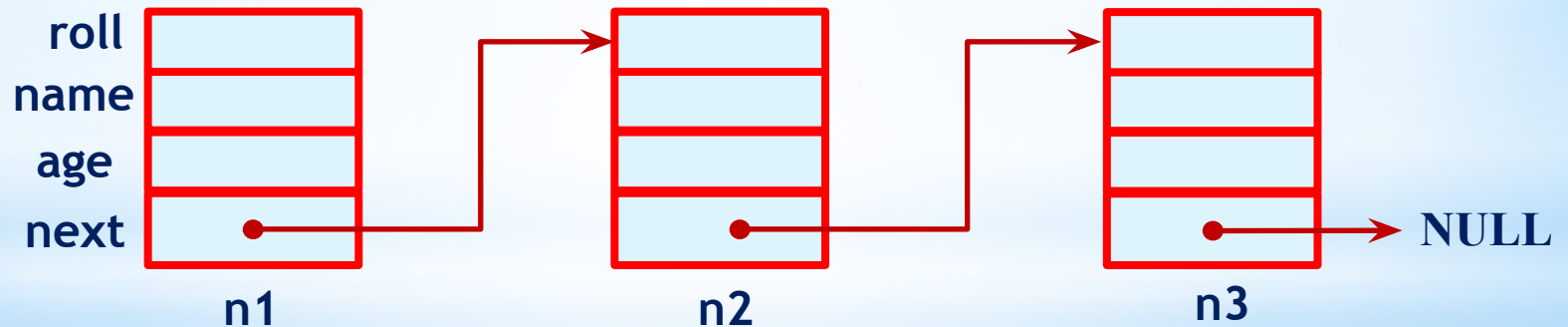
```
struct stud n1, n2, n3;
```

Example 1: Illustration

To create the links between nodes, it is written as:

```
n1.next = &n2 ;  
n2.next = &n3 ;  
n3.next = NULL ;    /* No more nodes follow */
```

- Now the list looks like:



Example 2: Creating a Single Linked List

C-program to store 10 values on a linked list reading the data from keyboard.

```
#include <stdio.h>
#include <stdlib.h>

struct node {
    int data;           //Data part
    struct node *next;  //Address part
}*header;

void createList(int n);    /* Functions to create a list*/

int main()
{
    int n;
    printf("Enter the total number of nodes: ");
    scanf("%d", &n);
    createList(n);
    return 0;
}
```

Example 2: Creating a Single Linked List

```
void createList(int n)
{
    struct node *newNode, *temp;
    int data, i;

    /* A node is created by allocating memory to a structure */
    newNode = (struct node *)malloc(sizeof(struct node));

    /* If unable to allocate memory for head node */
    if(newNode == NULL)
    {
        printf("Unable to allocate memory.");
    }
    else
    {
        printf("Enter the data of node 1: ");
        scanf("%d", &data);

        newNode->data = data; //Links the data field with data
        newNode->next = NULL; //Links the address field to NULL
        header = newNode;    //Header points to the first node
        temp = newNode;      //First node is the current node
    }
}
```

Example 2: Creating a Single Linked List

```
for(i=2; i<= n; i++)
{
    /* A newNode is created by allocating memory */
    newNode = (struct node *)malloc(sizeof(struct node));

    if(newNode == NULL)
    {
        printf("Unable to allocate memory.");
        break;
    }
    else
    {
        printf("Enter the data of node %d: ", i);
        scanf("%d", &data);

        newNode->data = data; //Links the data field of newNode with data
        newNode->next = NULL; //Links the address field of newNode with
NULL

        temp->next = newNode; //Links previous node i.e. temp to the
newNode
        temp = temp->next;
    }
}
```

Example 2: Illustration

- To start with, we have to create a **node** (the first node), and make **header** point to it.

```
newNode = (struct node *)malloc(sizeof(struct node));  
newNode->data = data;           //Links the data field with data  
newNode->next = NULL;          //Links the address field to NULL  
header = newNode;  
temp = newNode;
```

It creates a single node. For example, if the data entered is 100 then the list look like



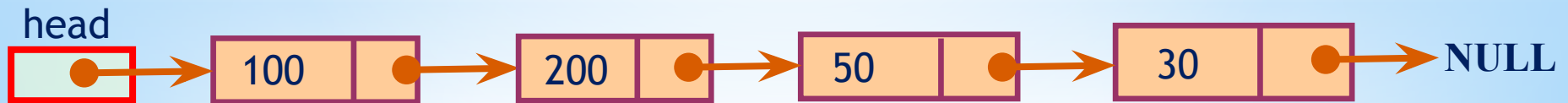
Creating a single linked list

If we need **n number of nodes** in the linked list:

- Allocate n newNodes, one by one.
- Read in the data for the newNodes.
- Modify the links of the newNodes so that the chain is formed.

```
newNode = (struct node *)malloc(sizeof(struct node));  
newNode->data = data;           //Links the data field of newNode with data  
newNode->next = NULL;          //Links the address field of newNode with NULL  
  
temp->next = newNode;          //Links previous node i.e. temp to the newNode  
temp = temp->next;
```

It creates **n** number of nodes . For e.g. if the data entered is 200, 50, 30 then the list look like



Example 3: Creating a SLL-C++

C-program to copy an array to a single linked list.

```
#include <iostream>

// Define the structure for a node
struct Node {
    int data;
    Node* next;
};

// Global header pointer (matching your original C design)

Node* header = nullptr;

// Function prototypes
void createList(int n);
void displayList();

int main()
{
    int n;
    std::cout << "Enter the total number of nodes: ";
    std::cin >> n;

    createList(n);
    displayList();

    return 0;
}
```

Example 2: Creating a SLL-C++

```
void createList(int n)
{
    // Handle edge case for zero or negative input
    if (n <= 0) return;
    int data;
    Node* temp = nullptr;

    // Create the first node
    std::cout << "Enter the data of node 1: ";
    std::cin >> data;
    header = new Node();
    header->data = data;
    header->next = nullptr;
    temp = header;

    // Create subsequent nodes
    for (int i = 2; i <= n; i++)
    {
        std::cout << "Enter the data of node " << i << ": ";
        std::cin >> data;

        Node* newNode = new Node();
        newNode->data = data;
        newNode->next = nullptr;
        temp->next = newNode;
        temp = temp->next;
    }
}
```

Example 2: Creating a SLL-C++

```
displayList()
{
    if (header == nullptr)
    {
        std::cout << "\nList is empty.\n";
        return;
    }

    Node* temp = header;
    std::cout << "\nData stored in the linked list:\n";

    while (temp != nullptr)
    {
        std::cout << "Data = " << temp->data << "\n";
        temp = temp->next;
    }
}
```

Operations on Linked Lists

Operations on single linked list

- Traversing a list
 - Printing, finding minimum, etc.
- Insertion of a node into a list
 - At front, end and anywhere, etc.
- Deletion of a node from a list
 - At front, end and anywhere, etc.
- Comparing two linked lists
 - Similarity, intersection, etc.
- Merging two linked lists into a larger list
 - Union, concatenation, etc.
- Ordering a list
 - Reversing, sorting, etc.

Traversing a Linked List

Single Linked List: Traversing

Once the linked list has been constructed and **header** points to the first node of the list,

- Follow the pointers.
- Display the contents of the nodes as they are traversed.
- Stop when the **next** pointer points to **NULL**.

The function **traverseList**(struct Node *) is given in the next slide. This function to be called from **main()** function as:

```
int main()
{
    // Assume header, the pointer to the linked list is given as an input
    printf("\n Data in the list \n");
    traverseList(header);
    return 0;
}
```

Single linked list: Traversing

```
void traverseList(struct Node *header)
{
    struct node *temp;

    /* If the list is empty i.e. head = NULL */
    if(header == NULL)
    {
        printf("List is empty.");
    }
    else
    {
        temp = header;
        while(temp != NULL)
        {
            printf("Data = %d\n", temp->data); //Prints the data of current node
            temp = temp->next;                //Advances the position of current node
        }
    }
}
```

Insertion in a Linked List

Single Linked List: Insertion

Insertion steps:

- Create a new node
- Start from the header node
- Manage links to
 - Insert at front
 - Insert at end
 - Insert at any position

```

#include <iostream>

struct Node {
    int data;
    Node* next;
};

// Fixed pool of nodes to avoid runtime dynamic allocation

const int MAX_NODES = 100;
Node nodePool[MAX_NODES];
int poolIndex = 0; // Tracks the next available node in the array pool

// Global header pointer
Node* header = nullptr;

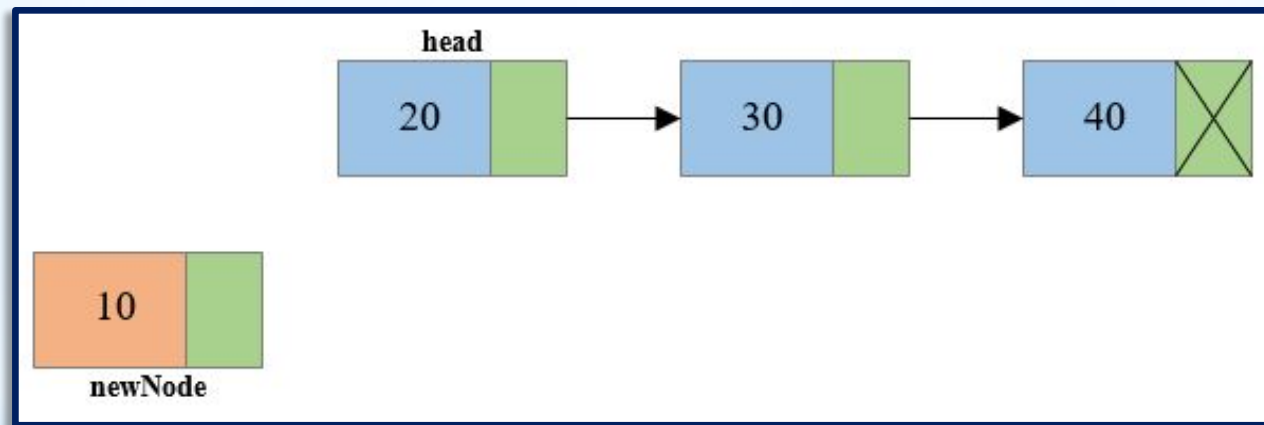
// Helper to "create a new node" from our static pool
Node* getNewNode(int value) {
    if (poolIndex >= MAX_NODES) {
        std::cout << "Error: Node pool is full!\n";
        return nullptr;
    }
    Node* newNode = &nodePool[poolIndex++];
    newNode->data = value;
    newNode->next = nullptr;
    return newNode;
}

```

Insertion at Front

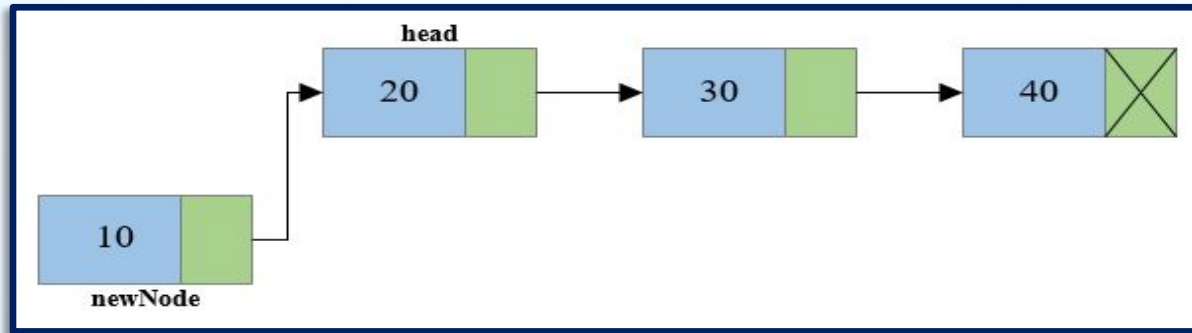
Steps to insert node at the beginning of singly linked list

Step 1: Create a new node.

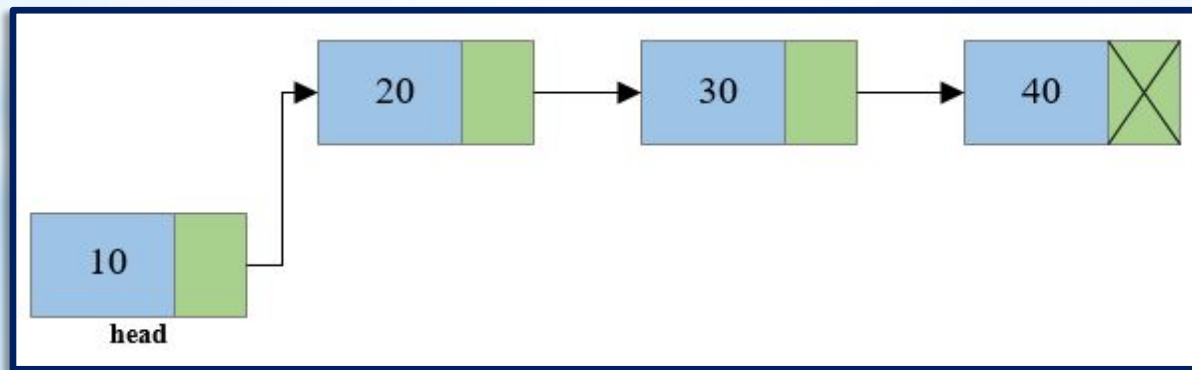


Insertion at Front

Step 2: Link the newly created node with the head node, i.e. the **newNode** will now point to **head** node.



Step 3: Make the new node as the head node, i.e. now **head** node will point to **newNode**.



Insertion at front

```
// 1. Insert at Front
void insertAtFront(int value) {
    Node* newNode = getNewNode(value);
    if (newNode == nullptr) return;

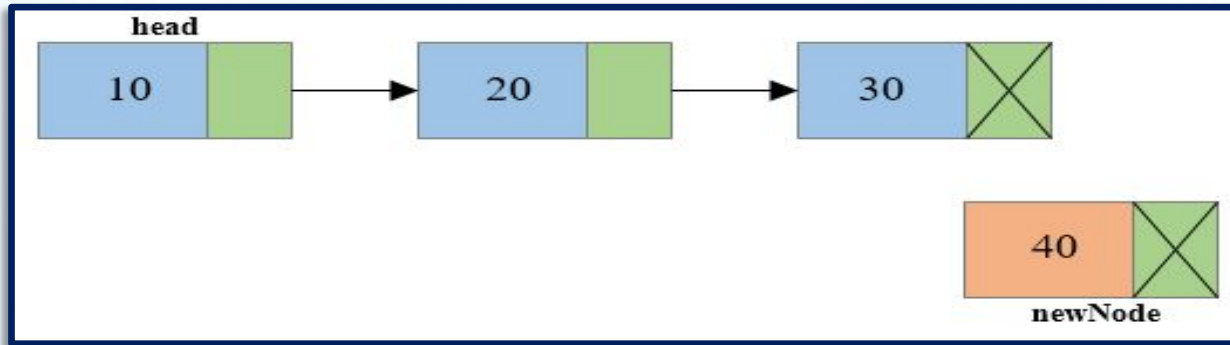
    newNode->next = header; // New node points to current first node
    header = newNode;      // Header now points to this new node
}

// Start from the header node and traverse to the end
Node* temp = header;
while (temp->next != nullptr) {
    temp = temp->next;
}
temp->next = newNode; // Link old end node to new node
}
```

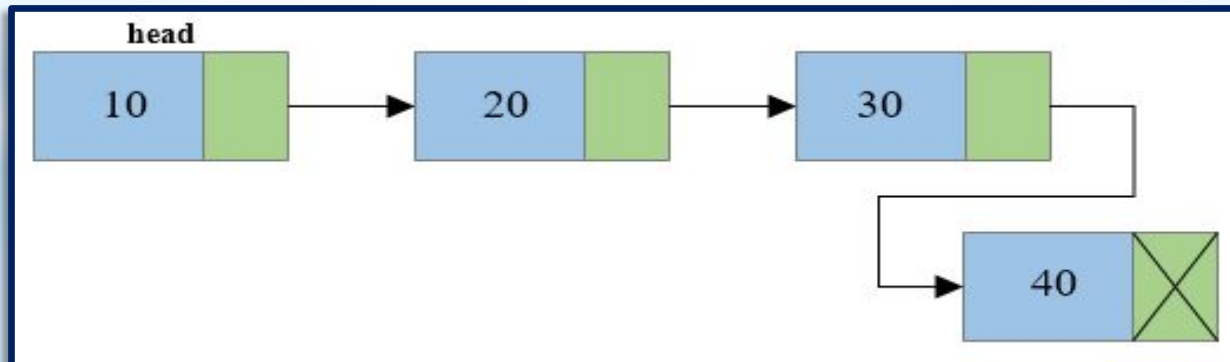
Single Linked List: Insertion at End

Steps to insert node at the end of Singly linked list

Step 1: Create a new node and make sure that the address part of the new node points to NULL. i.e. `newNode->next=NULL`



Step 2: Traverse to the last node of the linked list and connect the last node of the list with the new node, i.e. last node will now point to new node. (`lastNode->next = newNode`).



Insertion at End

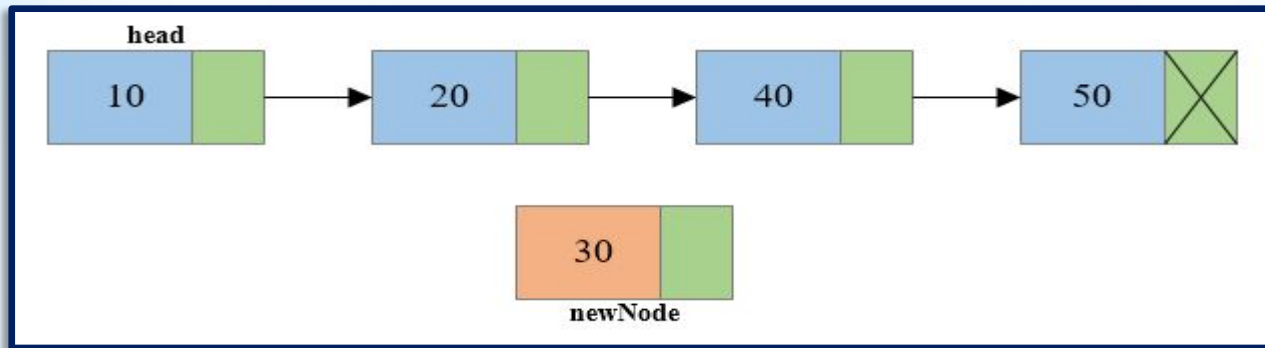
```
// 2. Insert at End
void insertAtEnd(int value) {
    Node* newNode = getNewNode(value);
    if (newNode == nullptr) return;

    // If list is empty, make it the first node
    if (header == nullptr) {
        header = newNode;
        return;
    }
}
```

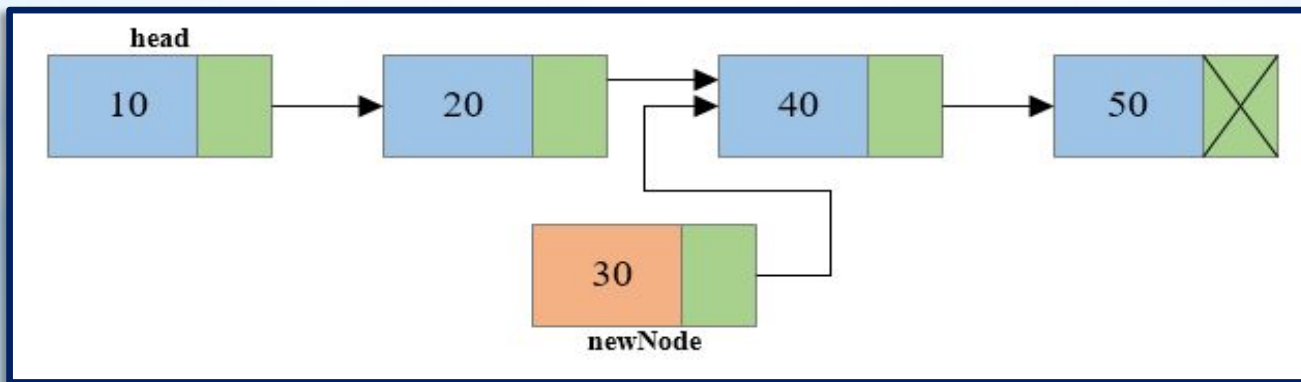
Single Linked List: Insertion at any Position

Steps to insert node at any position of Singly Linked List

Step 1: Create a new node.

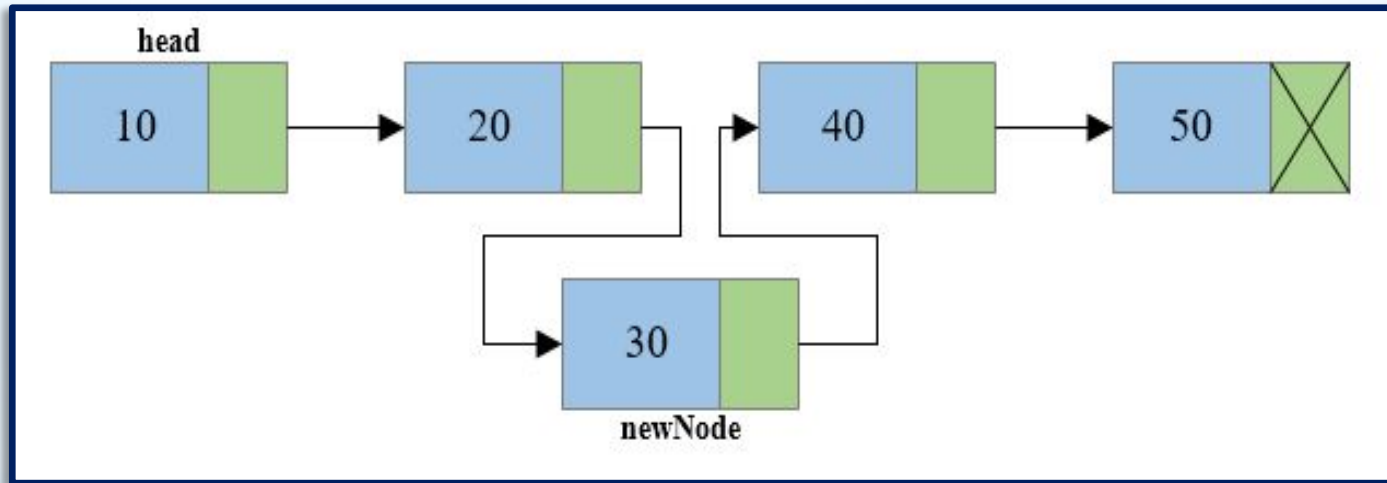


Step 2: Traverse to the $n-1^{\text{th}}$ position of the linked list and connect the new node with the $n+1^{\text{th}}$ node. ($\text{newNode} \rightarrow \text{next} = \text{temp} \rightarrow \text{next}$) where temp is the $n-1^{\text{th}}$ node.



Single Linked List: Insertion at any position

Step 3: Now at last connect the $n-1^{\text{th}}$ node with the new node i.e. the $n-1^{\text{th}}$ node will now point to new node. (`temp->next = newNode`) where temp is the $n-1^{\text{th}}$ node.



Insertion at any Position

```
// 3. Insert at Any Position (1-indexed position)
void insertAtAnyPosition(int value, int position) {
    if (position < 1) {
        std::cout << "Invalid position!\n";
        return;
    }

    if (position == 1) {
        insertAtFront(value);
        return;
    }

    Node* newNode = getNewNode(value);
    if (newNode == nullptr) return;

    // Start from the header node
    Node* temp = header;
    for (int i = 1; i < position - 1 && temp != nullptr; i++) {
        temp = temp->next;
    }

    // If position is out of bounds
    if (temp == nullptr) {
        std::cout << "Position out of range!\n";
        return;
    }

    // Manage links to insert
    newNode->next = temp->next;
    temp->next = newNode;
}
```

Insertion at any Position

```
// Function to print the stored values
void displayList() {
    if (header == nullptr) {
        std::cout << "List is empty.\n";
        return;
    }

    Node* temp = header;
    std::cout << "Linked List: ";
    while (temp != nullptr) {
        std::cout << temp->data << " -> ";
        temp = temp->next;
    }
    std::cout << "NULL\n";
}
```

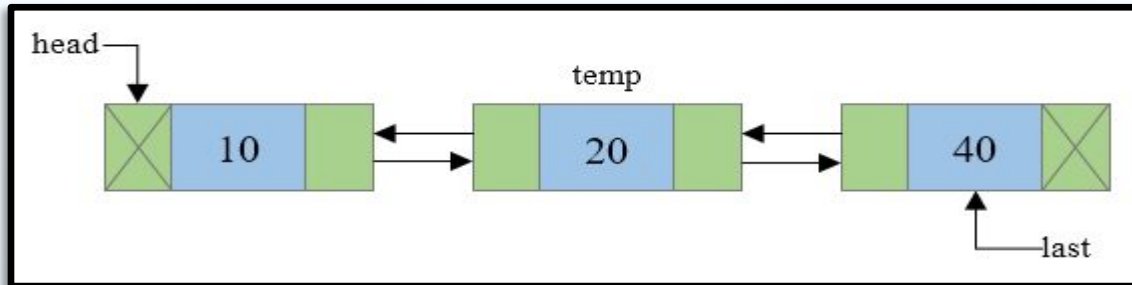
Insertion at any Position

```
int main() {  
    // Demonstration of all three insertion rules  
    std::cout << "--- Inserting at Front ---\n";  
    insertAtFront(20);  
    insertAtFront(10);  
    displayList(); // Output: 10 -> 20 -> NULL  
  
    std::cout << "\n--- Inserting at End ---\n";  
    insertAtEnd(40);  
    insertAtEnd(50);  
    displayList(); // Output: 10 -> 20 -> 40 -> 50 -> NULL  
  
    std::cout << "\n--- Inserting at Position 3 ---\n";  
    insertAtAnyPosition(30, 3);  
    displayList(); // Output: 10 -> 20 -> 30 -> 40 -> 50 -> NULL  
  
    return 0;  
}
```

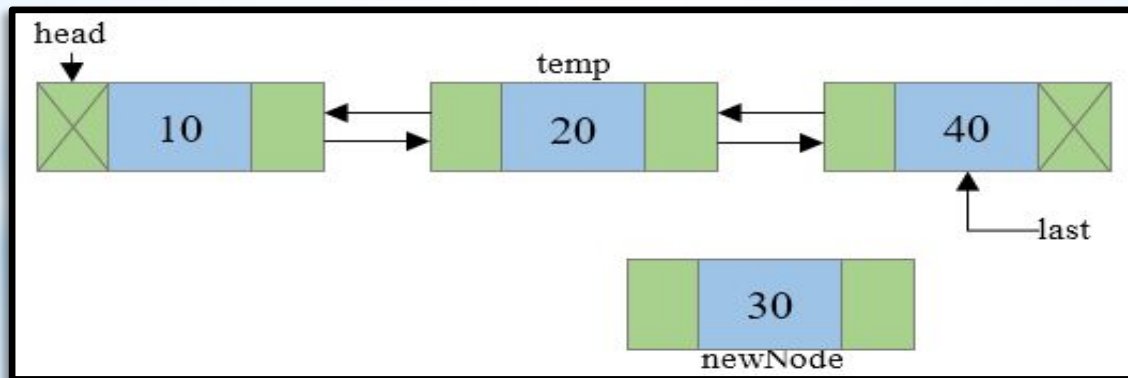
Double Linked List: Insertion at any Position

Steps to insert a new node at n^{th} position in a Doubly linked list.

Step 1: Traverse to $N-1$ node in the list, where N is the position to insert. Say **temp** now points to $N-1^{\text{th}}$ node.

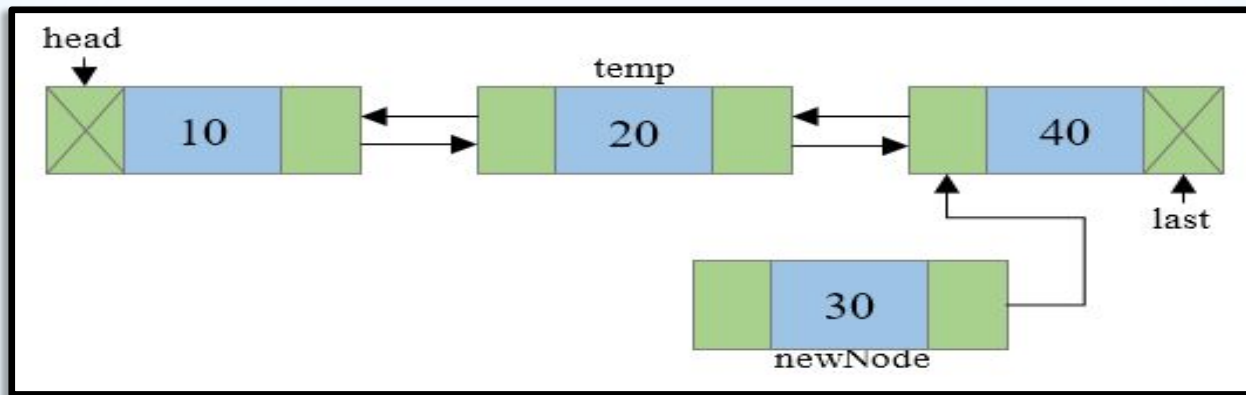


Step 2: Create a **newNode** that is to be inserted and assign some data to its data field.

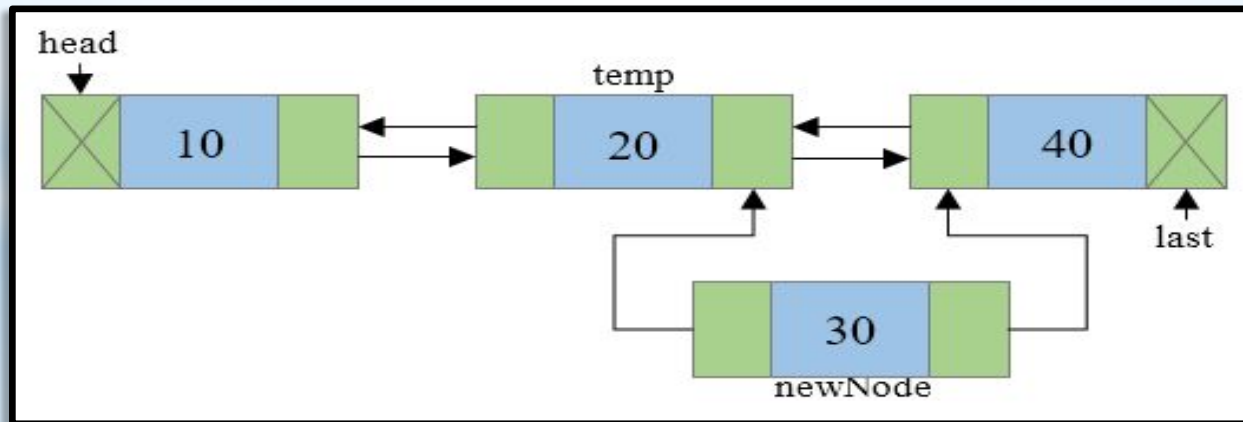


Doubly Linked List: Insertion at any Position

Step 3: Connect the next address field of **newNode** with the node pointed by next address field of **temp** node.

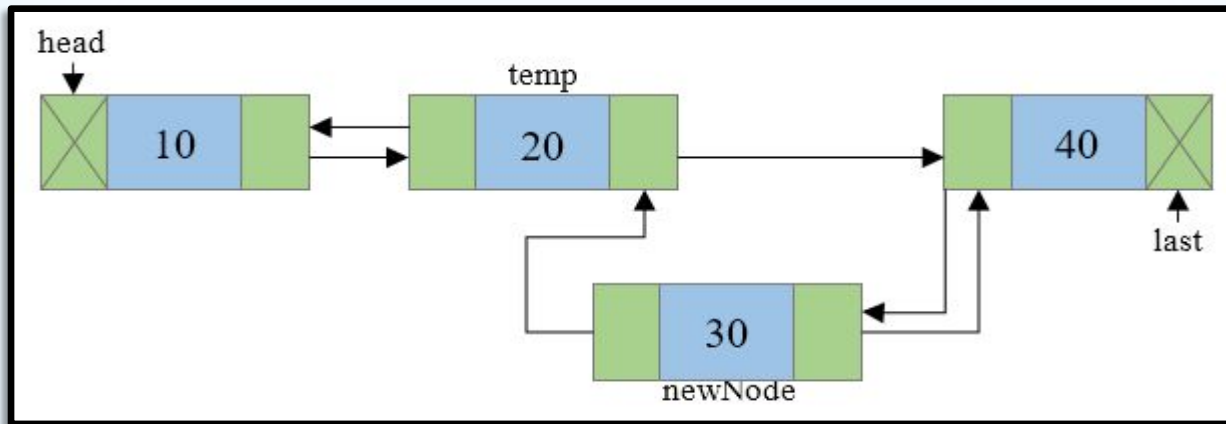


Step 4: Connect the previous address field of **newNode** with the **temp** node.

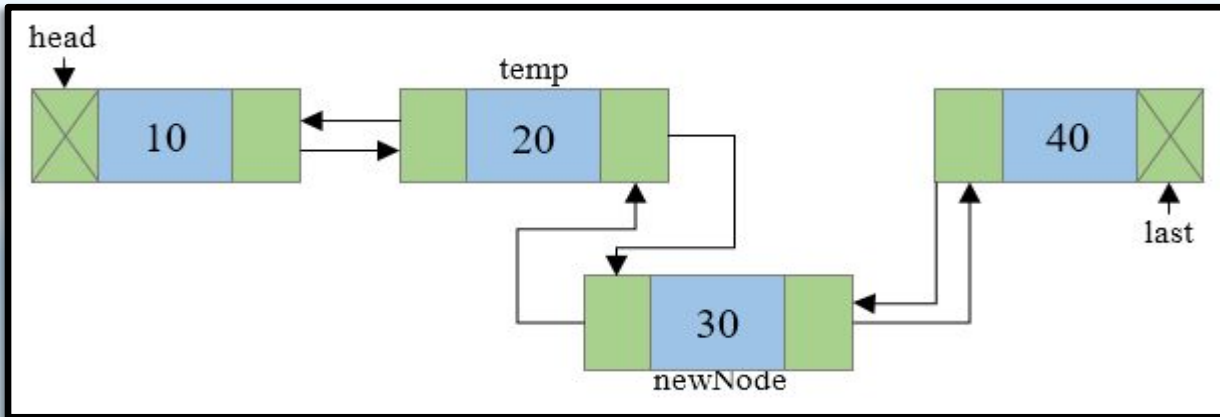


Doubly Linked List: Insertion at any Position

Step 5: Check if `temp.next` is not NULL then, connect the previous address field of node pointed by `temp.next` to `newNode`.

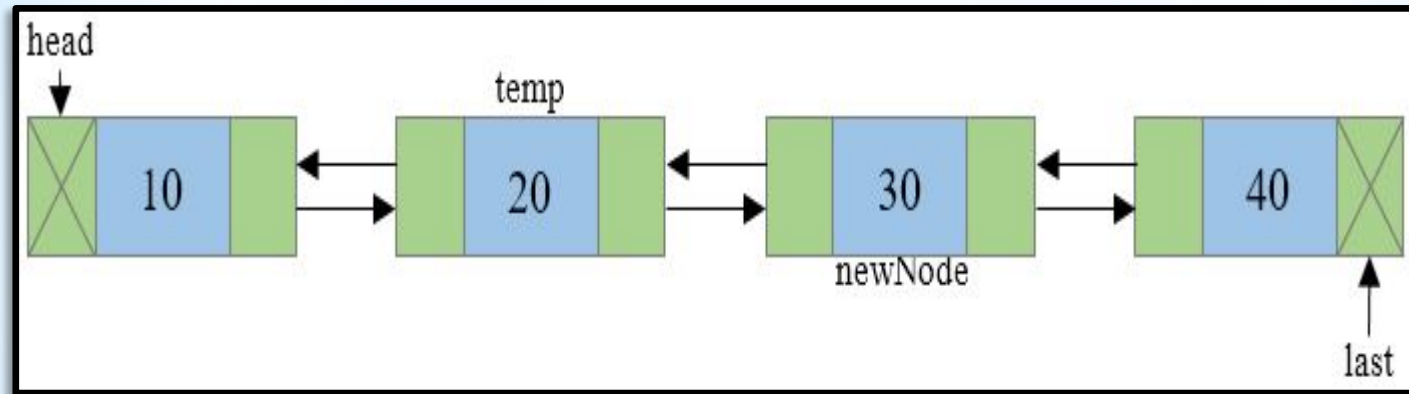


Step 6: Connect the next address field of `temp` node to `newNode`.



Doubly Linked List: Insertion at any Position

Step 7: Final doubly linked list looks like



Doubly Linked List: Insertion at any Position

```
#include <iostream>

struct Node {
    int data;
    Node* next; // Pointer to next node
    Node* prev; // Pointer to previous node
};

// Fixed pool of nodes to completely avoid runtime dynamic heap allocation
const int MAX_NODES = 100;
Node nodePool[MAX_NODES];
int poolIndex = 0; // Tracks the next available static element in the pool array

// Global header pointer
Node* header = nullptr;

// Helper function: Simulate "creating a new node" from our static pool array
Node* getNewNode(int value) {
    if (poolIndex >= MAX_NODES) {
        std::cout << "Error: Node pool is completely full!\n";
        return nullptr;
    }
    Node* newNode = &nodePool[poolIndex++];
    newNode->data = value;
    newNode->next = nullptr;
    newNode->prev = nullptr;
    return newNode;
}
```

Doubly Linked List: Insertion at any Position

```
// ===== INSERTION OPERATIONS =====

// 1. Insert at Front
void insertAtFront(int value) {
    Node* newNode = getNewNode(value);
    if (newNode == nullptr) return;

    if (header == nullptr) {
        header = newNode;
        return;
    }

    newNode->next = header;
    header->prev = newNode; // Old head now links backward to the new node
    header = newNode;      // Header now points to this new node
}

// 2. Insert at End
void insertAtEnd(int value) {
    Node* newNode = getNewNode(value);
    if (newNode == nullptr) return;

    if (header == nullptr) {
        header = newNode;
        return;
    }

    // Start from header and traverse to the absolute end node
    Node* temp = header;
    while (temp->next != nullptr) {
        temp = temp->next;
    }
}
```

Doubly Linked List: Insertion at any Position

```
// 3. Insert at Any Given Position (1-indexed)
void insertAtAnyPosition(int value, int position) {
    if (position < 1) {
        std::cout << "Invalid position target!\n";
        return;
    }

    if (position == 1) {
        insertAtFront(value);
        return;
    }

    // Start from the header node
    Node* temp = header;
    for (int i = 1; i < position - 1 && temp != nullptr; i++) {
        temp = temp->next;
    }

    if (temp == nullptr) {
        std::cout << "Position out of range!\n";
        return;
    }

    Node* newNode = getNewNode(value);
    if (newNode == nullptr) return;

    // Manage multiple directional link pointers to insert smoothly
    newNode->next = temp->next;
    newNode->prev = temp;

    if (temp->next != nullptr) {
        temp->next->prev = newNode; // Next node points backward to our new node
    }
}
```

Doubly Linked List: Insertion at any Position

```
// ===== DELETION OPERATIONS =====

// 1. Delete from Front
void deleteFromFront() {
    if (header == nullptr) {
        std::cout << "Underflow Error: List is empty.\n";
        return;
    }

    header = header->next;
    if (header != nullptr) {
        header->prev = nullptr; // New head's backward pointer must be null
    }
}

// 2. Delete from End
void deleteFromEnd() {
    if (header == nullptr) {
        std::cout << "Underflow Error: List is empty.\n";
        return;
    }

    if (header->next == nullptr) {
        header = nullptr;
        return;
    }

    // Start from the header node and step through to the last element
    Node* temp = header;
    while (temp->next != nullptr) {
        temp = temp->next;
    }
}
```

Doubly Linked List: Insertion at any Position

```
// 3. Delete from Any Given Position (1-indexed)
void deleteFromPosition(int position) {
    if (header == nullptr || position < 1) {
        std::cout << "Invalid delete action or empty list.\n";
        return;
    }

    if (position == 1) {
        deleteFromFront();
        return;
    }

    // Start from the header node and target the exact node to be deleted
    Node* temp = header;
    for (int i = 1; i < position && temp != nullptr; i++) {
        temp = temp->next;
    }

    if (temp == nullptr) {
        std::cout << "Target position out of range!\n";
        return;
    }

    // Bridge links around the node being deleted
    if (temp->next != nullptr) {
        temp->next->prev = temp->prev;
    }
    if (temp->prev != nullptr) {
        temp->prev->next = temp->next;
    }
}
```

Doubly Linked List: Insertion at any Position

```
// ===== DISPLAY/TRAVERSAL OPERATIONS =====

// Forward Display
void displayForward() {
    if (header == nullptr) {
        std::cout << "List is empty.\n";
        return;
    }

    Node* temp = header;
    std::cout << "Forward Display:  NULL <- ";
    while (temp != nullptr) {
        std::cout << temp->data;
        if (temp->next != nullptr) std::cout << " <=> ";
        temp = temp->next;
    }
    std::cout << " -> NULL\n";
}
```

Doubly Linked List: Insertion at any Position

```
// Backward Display
void displayBackward() {
    if (header == nullptr) {
        std::cout << "List is empty.\n";
        return;
    }

    // First go to the end of the list
    Node* temp = header;
    while (temp->next != nullptr) {
        temp = temp->next;
    }

    // Traverse backwards using the prev pointer
    std::cout << "Backward Display: NULL <- ";
    while (temp != nullptr) {
        std::cout << temp->data;
        if (temp->prev != nullptr) std::cout << " <=> ";
        temp = temp->prev;
    }
    std::cout << " -> NULL\n";
}
```

Doubly Linked List: Insertion at any Position

```
// ===== MAIN EXECUTION =====

int main() {
    std::cout << "=== RUNNING ALL DOUBLY LINKED LIST OPERATIONS ===\n\n";

    // 1. Showcase Insertions
    std::cout << "Inserting 20 then 10 at Front...\n";
    insertAtFront(20);
    insertAtFront(10);
    displayForward(); // 10 <=> 20

    std::cout << "\nInserting 40 then 50 at End...\n";
    insertAtEnd(40);
    insertAtEnd(50);
    displayForward(); // 10 <=> 20 <=> 40 <=> 50

    std::cout << "\nInserting 30 at Position 3...\n";
    insertAtAnyPosition(30, 3);
    displayForward(); // 10 <=> 20 <=> 30 <=> 40 <=> 50
    displayBackward(); // Verifies backward links: 50 <=> 40 <=> 30 <=> 20 <=> 10

    // 2. Showcase Deletions
    std::cout << "\nDeleting from Position 3 (Value 30)...\n";
    deleteFromPosition(3);
    displayForward(); // 10 <=> 20 <=> 40 <=> 50

    std::cout << "\nDeleting Front Node (Value 10)...\n";
    deleteFromFront();
    displayForward(); // 20 <=> 40 <=> 50

    std::cout << "\nDeleting End Node (Value 50)...\n";
    deleteFromEnd();
    displayForward(); // 10 <=> 20 <=> 40
```

Few Exercises to Try Out

For doubly linked list write a function to:

- Insert a node at front of the list and at end of the list.

```
insert_front(data);  
insert_end(data);
```

- Sort the DLL in ascending order.
- Count the number of nodes in the given DLL.

Deletion from a Linked List

Single Linked List: Deletion

Deletion steps

- Start from the header node
- Manage links to
 - Delete at front
 - Delete at end
 - Delete at any position
- freeingup the node as free space.

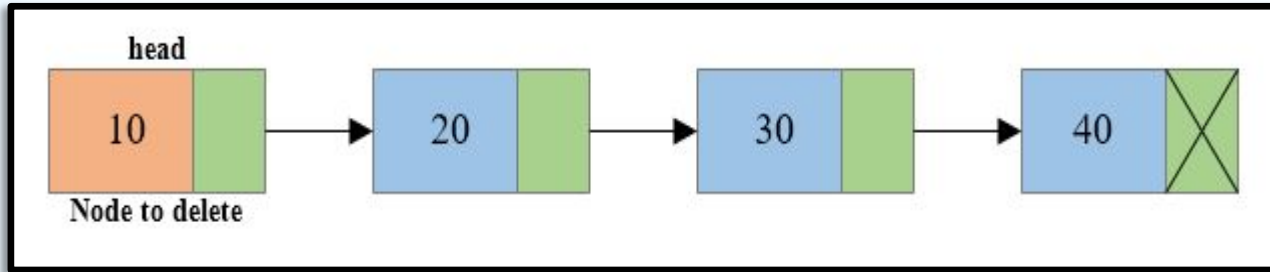
Free Memory after Deletion

- Do not forget to **free()** memory location dynamically allocated for a node **after deletion** of that node.
- It is the programmer's responsibility to free that memory block.
- Failure to do so may create a **dangling pointer** – a memory, that is not used either by the programmer or by the system.
- The content of a free memory is not erased until it is overwritten.

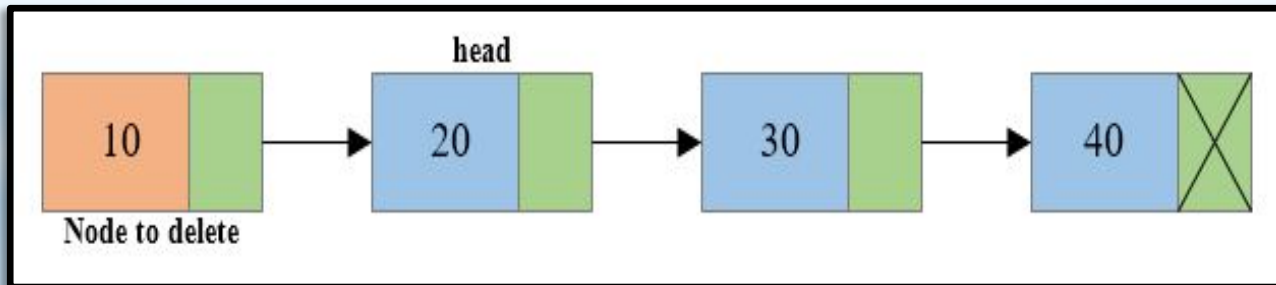
Single Linked List: Deletion at Front

Steps to delete first node of Singly Linked List

Step 1: Copy the address of first node i.e. **head** node to some temp variable say **toDelete**.

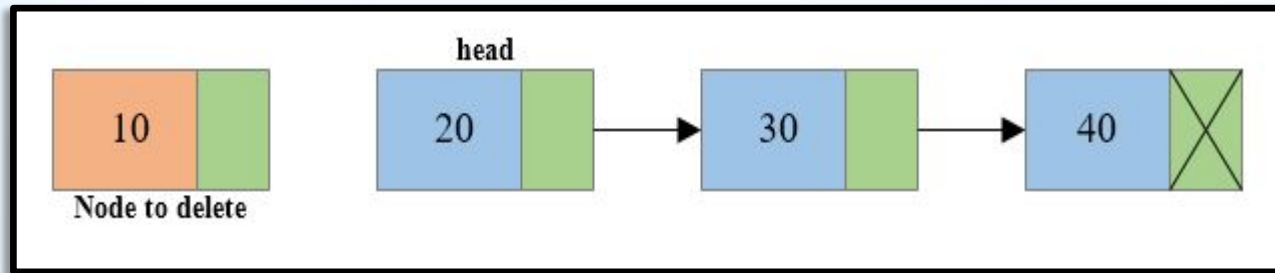


Step 2: Move the head to the second node of the linked list ($\text{head} = \text{head} \rightarrow \text{next}$).

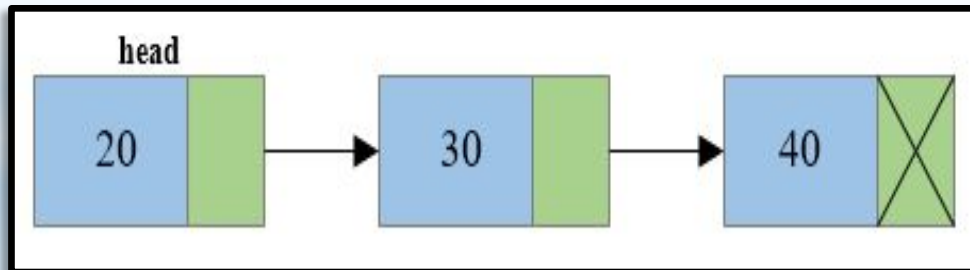


Single linked list: Deletion at front

Step 3: Disconnect the connection of first node to second node.



Step 4: Free the memory occupied by the first node.



Deletion at Front

```
/* Delete the first node of the linked list */
void deleteFirstNode()
{
    struct node *toDelete;

    if(head == NULL)
    {
        printf("List is already empty.");
    }
    else
    {
        toDelete = head;
        head = head->next;

        printf("\nData deleted = %d\n", toDelete->data);

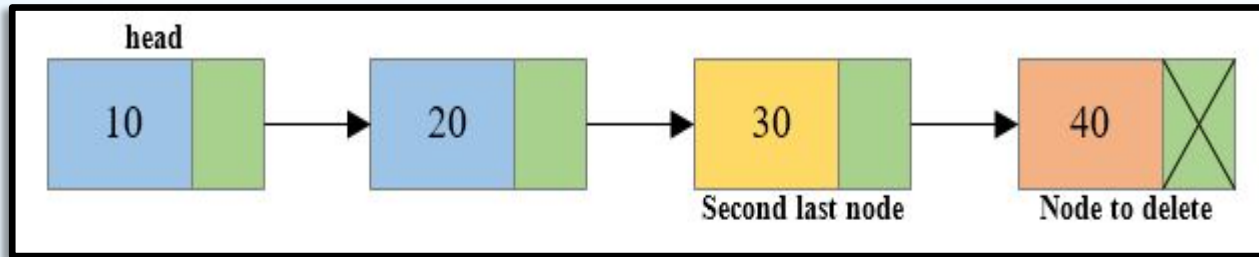
        /* Clears the memory occupied by first node*/
        free(toDelete);

        printf("SUCCESSFULLY DELETED FIRST NODE FROM LIST\n");
    }
}
```

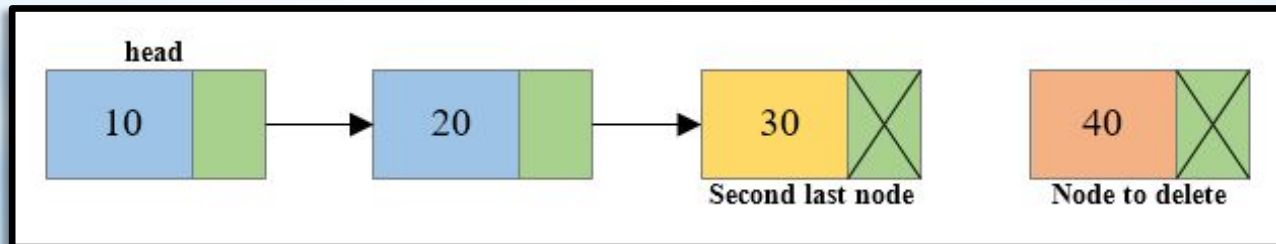
Single linked list: Deletion at End

Steps to delete last node of a Singly Linked List

Step 1: Traverse to the last node of the linked list keeping track of the second last node in some temp variable say **secondLastNode**.

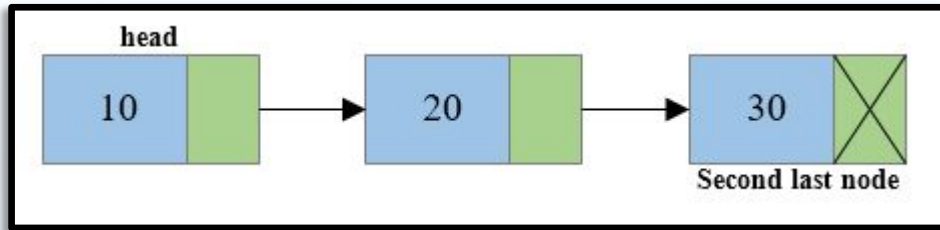


Step 2: If the last node is the head node then make the head node as NULL else disconnect the second last node with the last node i.e. `secondLastNode->next = NULL`



Single linked list: Deletion at End

Step 3: Free the memory occupied by the last node.



Deletion at End

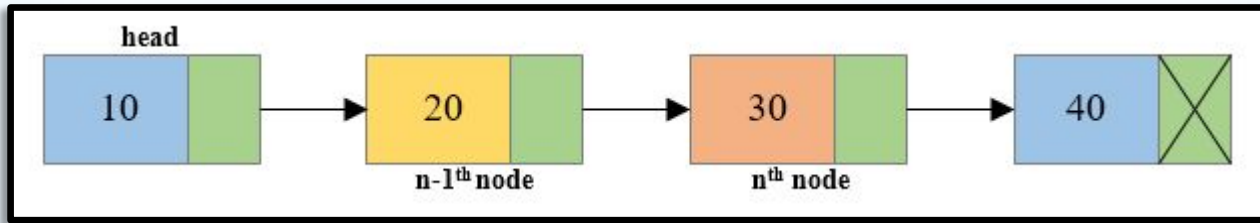
```
/* Delete the last node of the linked list */
void deleteLastNode()
{
    struct node *toDelete, *secondLastNode;
    toDelete = head;
    secondLastNode = head;

    while(toDelete->next != NULL) /* Traverse to the last node of the
list*/
    {
        secondLastNode = toDelete;
        toDelete = toDelete->next;
    }
    if(toDelete == head)
    {
        head = NULL;
    }
    else
    {
        /* Disconnects the link of second last node with last node */
        secondLastNode->next = NULL;
    }
    /* Delete the last node */
    free(toDelete);
}
```

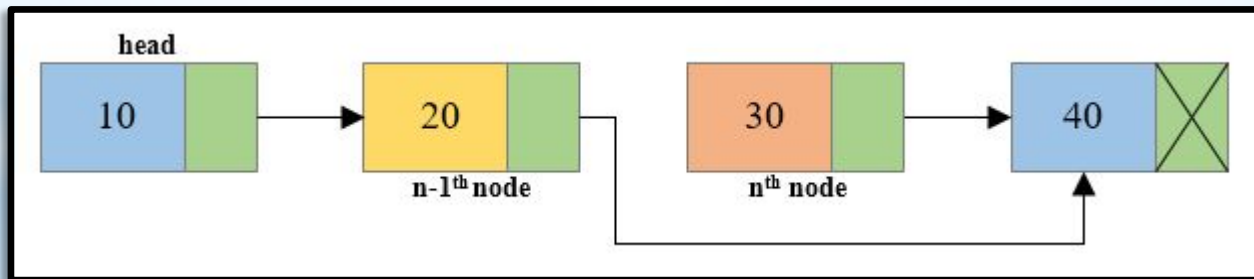
Single Linked List: Deletion at any Position

Steps to delete a node at any position of Singly Linked List

Step 1: Traverse to the n^{th} node of the singly linked list and also keep reference of $n-1^{\text{th}}$ node in some temp variable say **prevNode**.

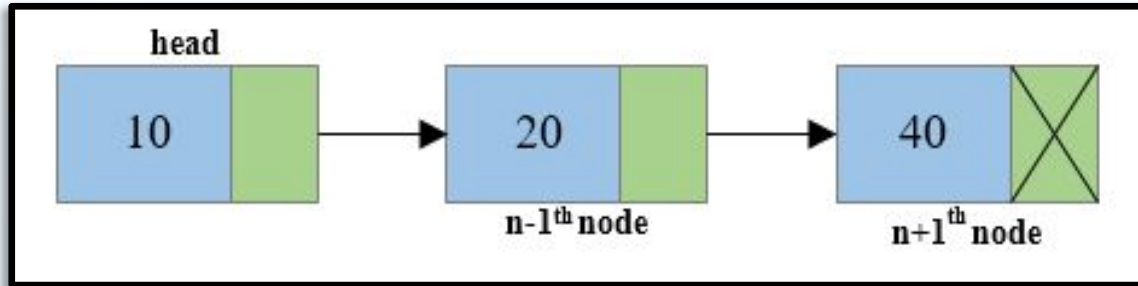


Step 2: Reconnect $n-1^{\text{th}}$ node with the $n+1^{\text{th}}$ node i.e. `prevNode->next = toDelete->next` (Where **prevNode** is $n-1^{\text{th}}$ node and **toDelete** node is the n^{th} node and `toDelete->next` is the $n+1^{\text{th}}$ node).



Single Linked List: Deletion at any Position

Step 3: Free the memory occupied by the n^{th} node i.e. **toDelete** node.



Deletion at any Position

```
/* Delete the node at any given position of the linked list */
void deleteMiddleNode(int position)
{
    int i;
    struct node *toDelete, *prevNode;
    if(head == NULL)
    {
        printf("List is already empty.");
    }
    else
    {
        toDelete = head;
        prevNode = head;

        for(i=2; i<=position; i++)
        {
            prevNode = toDelete;
            toDelete = toDelete->next;

            if(toDelete == NULL)
                break;
        }
    }
}
```

Deletion at any Position

```
if(toDelete != NULL)
{
    if(toDelete == head)
        head = head->next;

    prevNode->next = toDelete->next;
    toDelete->next = NULL;

    /* Deletes the n node */
    free(toDelete);

    printf("SUCCESSFULLY DELETED NODE FROM MIDDLE OF LIST\n");
}
else
{
    printf("Invalid position unable to delete.");
}
}
```

Comparing Two Linked Lists

Single Linked List: Comparing two Lists

Comparing two linked list includes

- Identifying whether the given two linked list are **identical**.
- Two Linked Lists are identical when they have **same data and arrangement** of data is also same.
- Checking whether the lists have **same values** when the **arrangement** is not same.

Comparing two Linked Lists

```
/* Return true if linked lists a and b are identical, otherwise false */
bool areIdentical(struct node *a, struct node *b)
{
    while (a != NULL && b != NULL)
    {
        if (a->data != b->data)
            return false;
        /* If we reach here, then a and b are not NULL and their data is same,
           so move to next nodes in both lists */
        a = a->next;
        b = b->next;
    }
    //If linked lists are identical, then 'a' and 'b' must be NULL at this
    point.
    return (a == NULL && b == NULL);
}

int main()
{
    struct node *a, *b;
    a = createList(5); // e.g: a: 5->4->3->2->1
    b = createList(5); // e.g: b: 5->4->3->2->1
    areIdentical(a, b)? printf("Identical"): printf("Not identical");
    return 0;
}
```

Few Exercises to Try Out

Write a function to:

- Concatenate or merge two given list into one big list.
`node *concatenate (node *a, node *b) ;`
- Compare two given list with same data but different arrangement.
e.g: a: 5->4->3->2->1
 b: 1->2->3->4->5
- Count the number of nodes in the given list using iterative method and recursive method.

Ordering Linked List

Single Linked List: Reversing

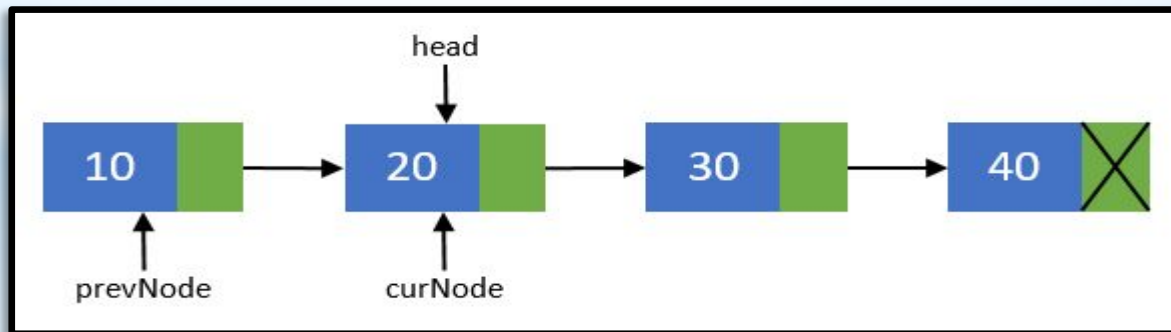
Reversing a list can be performed in two ways:

- **Iterative** method
- **Recursive** method

Steps to reverse a Singly Linked List using Iterative method

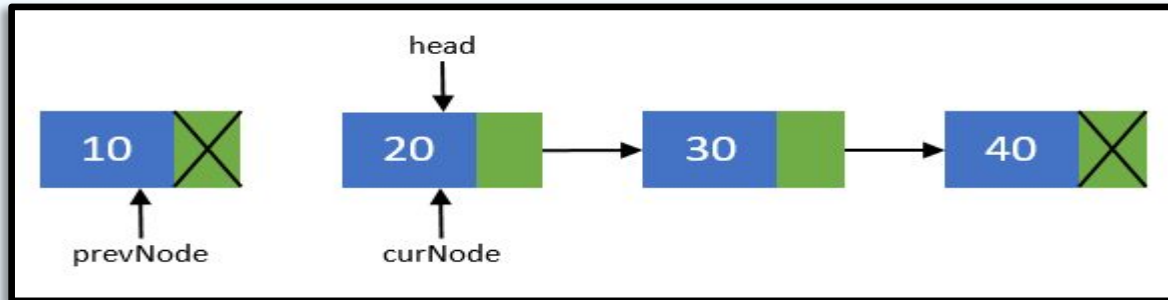
Step 1: Create two more pointers other than **head** namely **prevNode** and **curNode** that will hold the reference of previous node and current node respectively.

- Make sure that prevNode points to first node i.e. `prevNode = head`.
- head should now point to its next node i.e. `head = head->next`.
- curNode should also points to the second node i.e. `curNode = head`.

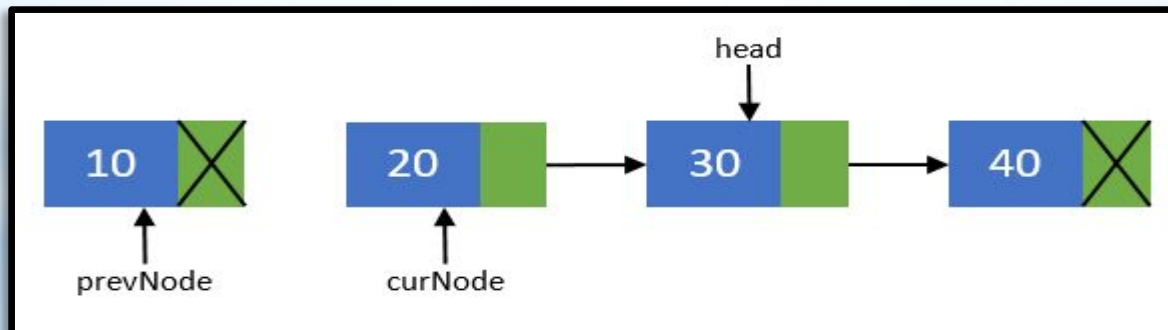


Reversing a List

Step 2: Now, disconnect the first node from others. We will make sure that it points to none. As this node is going to be our last node. Perform operation `prevNode->next = NULL`.



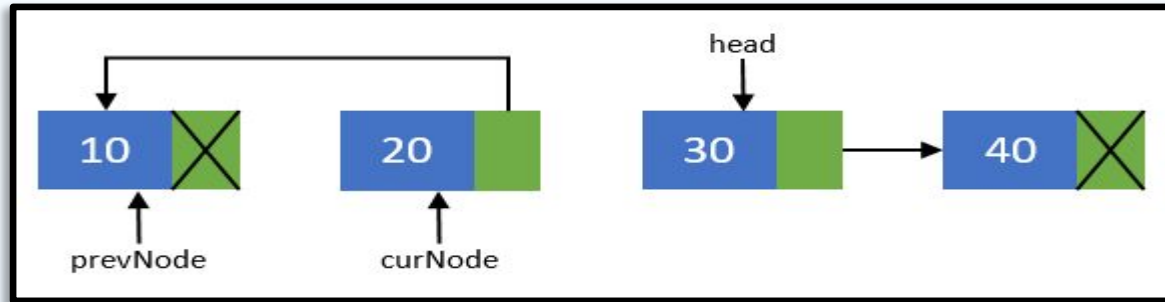
Step 3: Move the head node to its next node i.e. `head = head->next`.



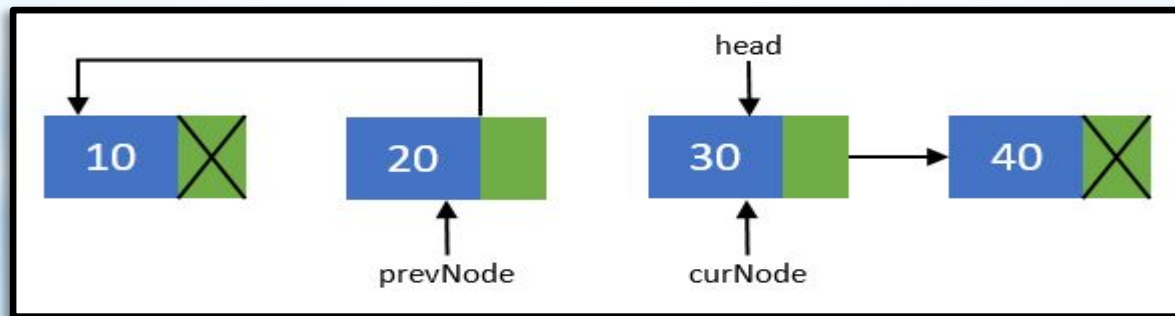
Reversing a List

Step 4: Now, re-connect the current node to its previous node

```
i.e. curNode->next = prevNode;
```



Step 5: Point the previous node to current node and current node to head node. Means they should now point to `prevNode = curNode;` and `curNode = head.`

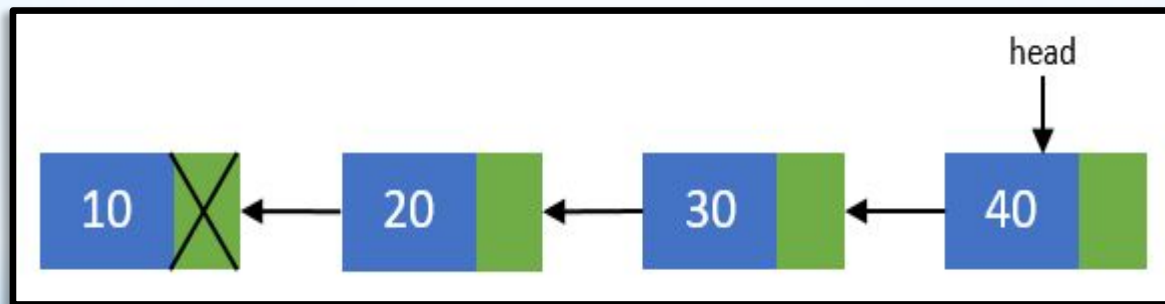


Reversing a List

Step 6: Repeat steps 3-5 till head pointer becomes **NULL**.

Step 7: Now, after all nodes has been re-connected in the reverse order. Make the last node as the first node. Means the head pointer should point to prevNode pointer.

- Perform `head = prevNode;` And finally you end up with a reversed linked list of its original.



Reversing a List: Iterative Method

```
void reverseList()
{
    struct node *prevNode, *curNode;
    if(head != NULL)
    {
        prevNode = head;
        curNode = head->next;
        head = head->next;

        prevNode->next = NULL;    //Makes the first node as last node

        while(head != NULL)
        {
            head = head->next;
            curNode->next = prevNode;

            prevNode = curNode;
            curNode = head;
        }

        head = prevNode;    //Makes the last node as head
        printf("SUCCESSFULLY REVERSED LIST\n");
    }
}
```

Reversing a List: Recursive Method

```
/* Function to reverse the linked list */
void Recursive_Reverse(struct node* head)
{
    if (head == NULL) // boundary condition to stop recursion
        return;

    Recursive_Reverse(head->next); // print the list after head node

    printf("%d  ", head->data); // After everything else is printed, print
head
}

/* It should be called from the main() function as */

int main()
{
    int n = 10;
    createList(n); // creates 10 nodes in the linked list
    Recursive_Reverse(head);

    return 0;
}
```

Single Linked List: Sorting

The linked list can be ordered using any of the following sorting algorithms:

- Insertion sort
- Selection sort
- Merge sort
- Quick sort
- Bubble sort, etc.

Here, we discuss **insertion** sort for ordering linked list.

Sorting a List using Insertion Sort

```
// function to sort a singly linked list using insertion sort
void insertionSort(struct node **head_ref)
{
    // Initialize sorted linked list
    struct node *sorted = NULL;

    // Traverse the given linked list and insert every node to be sorted
    struct node *current = *head_ref;
    while (current != NULL)
    {
        struct node *next = current->next;

        // insert current in sorted linked list
        sortedInsert(&sorted, current);

        current = next; // Update current
    }

    // Update head_ref to point to sorted linked list
    *head_ref = sorted;
}
```

Sorting a List using Insertion Sort

```
/* function to insert a new_node in a list.*/
void sortedInsert(struct node** head_ref, struct node* new_node)
{
    struct node* current;

    /* Special case for the head end */
    if (*head_ref == NULL || (*head_ref)->data >= new_node->data)
    {
        new_node->next = *head_ref;
        *head_ref = new_node;
    }
    else
    {
        /* Locate the node before the point of insertion */
        current = *head_ref;
        while (current->next!=NULL && current->next->data <
new_node->data)
        {
            current = current->next;
        }
        new_node->next = current->next;
        current->next = new_node;
    }
}
```

Sorting a List using Insertion Sort

```
int main()
{
    int n=5;
    createList(n);
    insertionSort(&head);
    printf("\nData after sorting the list \n");
    displayList();
    return 0;
}
```

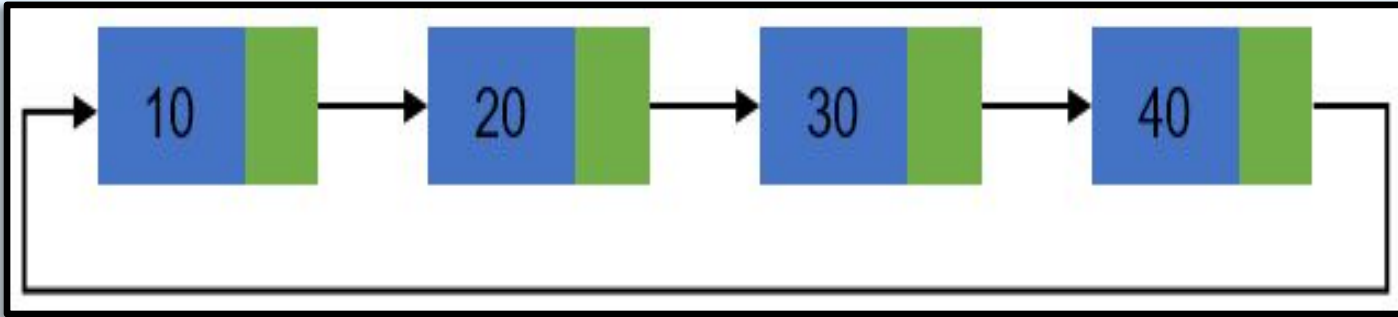
Output:

```
Enter the data of node 1: 6
Enter the data of node 2: 88
Enter the data of node 3: 42
Enter the data of node 4: 21
Enter the data of node 5: 1
SINGLY LINKED LIST CREATED SUCCESSFULLY

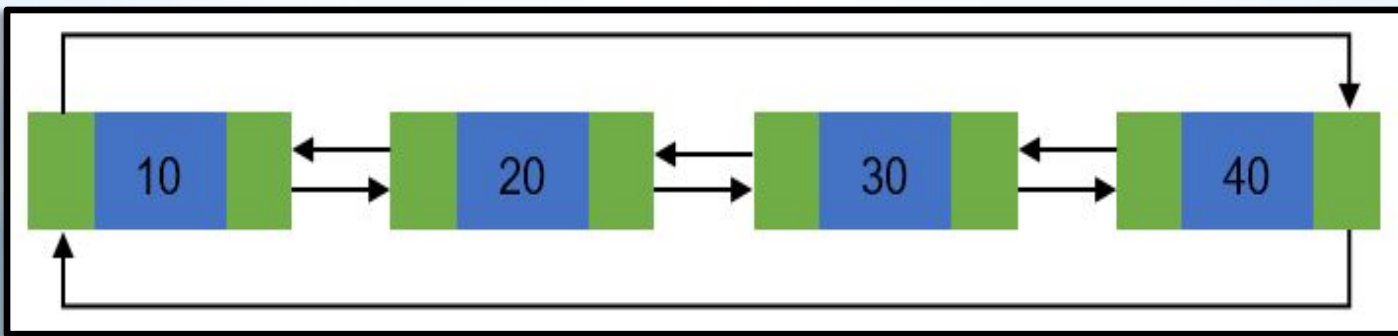
Data after sorting the list
Data = 1
Data = 6
Data = 21
Data = 42
Data = 88
```

Circular linked list:

Basic structure of singly circular linked list:



Doubly circular linked list:



Circular linked list:

Advantages of a Circular linked list

- Entire list can be traversed from any node.
- Circular lists are the required data structure when we want a list to be accessed in a circle or loop.
- Despite of being singly circular linked list we can easily traverse to its previous node, which is not possible in singly linked list.

Disadvantages of Circular linked list

- Circular list are complex as compared to singly linked lists.
- Reversing of circular list is a complex as compared to singly or doubly lists.
- If not traversed carefully, then we could end up in an infinite loop.
- Like singly and doubly lists circular linked lists also doesn't supports direct accessing of elements.

Operations on circular linked list

- Creation of list
- Traversal of list
- Insertion of node
 - At the beginning of list
 - At any position in the list
- Deletion of node
 - Deletion of first node
 - Deletion of node from middle of the list
 - Deletion of last node
- Counting total number of nodes
- Reversing of list

Creation and Traversal of a Circular List

```
#include <stdio.h>
#include <stdlib.h>

/* Basic structure of Node */

struct node {
    int data;
    struct node * next;
}*head;

int main()
{
    int n, data;
    head = NULL;

    printf("Enter the total number of nodes in list: ");
    scanf("%d", &n);
    createList(n);          // function to create circular linked list
    displayList();          // function to display the list

    return 0;
}
```

Circular Linked List: Creation of List

```
void createList(int n)
{
    int i, data;
    struct node *prevNode, *newNode;
    if(n >= 1){                                /* Creates and links the head node */
        head = (struct node *)malloc(sizeof(struct node));

        printf("Enter data of 1 node: ");
        scanf("%d", &data);

        head->data = data;
        head->next = NULL;
        prevNode = head;

        for(i=2; i<=n; i++){                    /* Creates and links rest of the n-1 nodes */
            newNode = (struct node *)malloc(sizeof(struct node));

            printf("Enter data of %d node: ", i);
            scanf("%d", &data);

            newNode->data = data;
            newNode->next = NULL;
            prevNode->next = newNode; //Links the previous node with newly created node
            prevNode = newNode;      //Moves the previous node ahead
        }

        prevNode->next = head; //Links the last node with first node
        printf("\nCIRCULAR LINKED LIST CREATED SUCCESSFULLY\n");
    }
}
```

Circular Linked List: Traversal of List

```
void displayList()
{
    struct node *current;
    int n = 1;

    if(head == NULL)
    {
        printf("List is empty.\n");
    }
    else
    {
        current = head;
        printf("DATA IN THE LIST:\n");

        do {
            printf("Data %d = %d\n", n, current->data);

            current = current->next;
            n++;
        }while(current != head);
    }
}
```

Few Exercises to Try Out

For circular linked list write a function to:

- Insert a node at any position of the list and delete from the beginning of the list.

```
insert_position(data, position) ;  
delete_front() ;
```

- Reverse the given circular linked link.

Any question?



You may post your question(s) at the “Discussion Forum” maintained in the course Web page.

Problems to Ponder...

1. Will be added shortly....

Problems for Practice...

***You can check the Moodle course management system for a set of problems for your own practice.**

- Login to the Moodle system at <http://cse.iitkgp.ac.in/>
- Select “PDS Spring-2017 (Theory) in the link “My Courses”
- Go to Topic 10: Practice Sheet #10 : Linked Lists

***Solutions to the problems in Practice Sheet #06 will be uploaded in due time.**

If you try to solve problems yourself, then you will learn many things automatically.

Spend few minutes and then enjoy the study.